Root Cause Analysis

🔴 Bug #1 — `api.ts`: Wrong `BASE_URL` for Mobile (CRITICAL)

File: `apps/mobile/src/services/api.ts`

```
// ❌ BROKEN — localhost on a physical device/emulator doesn't resolve to your dev machine
const BASE_URL = 'http://localhost:3000/api/v1';
```

On **Android emulators**, `localhost` refers to the emulator itself, not your dev machine. On **iOS simulators** it works, but on **physical devices** it fails entirely. This single line causes **every API call to fail silently**, which is why pressing Register/Login does nothing.

Fix:

```
// ☑ For Android Emulator
const BASE_URL = 'http://10.0.2.2:3000/api/v1';

// ☑ For physical device (use your machine's LAN IP)
// const BASE_URL = 'http://192.168.X.X:3000/api/v1';

// ☑ Best practice — use an env variable
const BASE_URL = process.env.EXPO_PUBLIC_API_URL ?? 'http://10.0.2.2:3000/api/v1';
```

🔴 Bug #2 — `api.ts`: `register()` Missing `tosAccepted` in Payload but Validation Rejects It

File: `apps/mobile/src/services/api.ts`

```
// ❌ BROKEN — sends tosAccepted to backend, but backend DTO has whitelist: true
// and forbidNonWhitelisted: true — any extra field THROWS a 400 error
async register(email, username, password, campus?, tosAccepted?) {
  return this.request('POST', '/auth/register', {
    email, username, password, campus, tosAccepted  // ← tosAccepted will be rejected!
  });
```

```
}
```

The `main.ts` has `forbidNonWhitelisted: true` in `ValidationPipe`. If `tosAccepted` is NOT declared in `RegisterDto`, this **immediately throws a 400 Bad Request** and the register call fails. Check your `RegisterDto` — if `tosAccepted` isn't declared there, every registration attempt dies here.

Fix — in `apps/backend/src/modules/auth/dto/auth.dto.ts`:

```
import { IsEmail, IsString, MinLength, IsOptional, IsBoolean } from 'class-validator';

export class RegisterDto {
  @IsEmail()
  email: string;

  @IsString()
  username: string;

  @IsString()
  @MinLength(8)
  password: string;

  @IsOptional()
  @IsString()
  displayName?: string;

  @IsOptional()
  @IsString()
  campus?: string;

  @IsOptional()
  @IsString()
  institution_id?: string;

  @IsOptional()
  @IsBoolean()
  tosAccepted?: boolean;  // ← MUST be here or strip it from api.ts
}

export class LoginDto {
  @IsEmail()
  email: string;

  @IsString()
```

```
  password: string;
}
```

🟢 Bug #3 — Campus Selection: `getCampuses` Called Before Domain is Valid

File: `apps/mobile/src/screens/auth/AuthScreen.tsx`

The `useEffect` fetches campuses when `email` changes and a `@domain` is present. But due to **Bug #1** (wrong BASE_URL), this API call silently fails — `campuses` stays `[]` — so the campus picker **never renders**, even for valid `.edu` emails. Once Bug #1 is fixed, this will automatically start working.

However there's a secondary issue: if `campuses.length === 0` (e.g., institution not in DB yet), registration proceeds without a campus, which then hits the `!institution` check in `auth.service.ts` and throws:

```
"Your college is not yet on BLITZR. You have been added to the waitlist."
```

...which surfaces as "Authentication Failed" to the user with no clear indication it's a campus/institution issue.

Fix — improve error messaging in `AuthScreen.tsx`:

```
} catch (error: any) {
  const msg = error.message || 'Unknown error';
  if (msg.includes('waitlist')) {
    Alert.alert('Campus Not Supported Yet', msg);
  } else if (msg.includes('already registered')) {
    Alert.alert('Account Exists', 'This email is already registered. Try logging in.');
  } else {
    Alert.alert('Authentication Failed', msg);
  }
}
```

🟢 Bug #4 — `useAuthStore` Has No Persistence

File: `apps/mobile/src/store/useAuthStore.ts`

The auth store uses plain Zustand with **no persistence**. Every time the app restarts, the user is logged out. This isn't what's blocking login right now, but it means once you fix the above bugs, users will be logged out on every reload.

Fix — add `zustand/middleware persist`:

```
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      isAuthenticated: false,
      userId: null,
      username: null,
      token: null,
      tosAccepted: false,
      login: (userId, username, token, tosAccepted) =>
        set({ isAuthenticated: true, userId, username, token, tosAccepted }),
      acceptTos: () => set({ tosAccepted: true }),
      logout: () => set({ isAuthenticated: false, userId: null, username: null, token: null }),
    }),
    {
      name: 'blitzr-auth',
      storage: createJSONStorage(() => AsyncStorage),
    }
  )
);
```

🟢 Bug #5 — `TosGuard` Blocks Auth Routes (Potential)

File: `apps/backend/src/app.module.ts`

The `TosGuard` is registered as a **global APP_GUARD**, meaning it runs on **every** route including `/auth/register` and `/auth/login`. If `TosGuard` checks `tos_accepted` and the user isn't authenticated yet, it may be **blocking unauthenticated auth requests**.

Fix — ensure `TosGuard` skips public auth routes:

```
// In tos.guard.ts — add a public route bypass
import { Reflector } from '@nestjs/core';
import { IS_PUBLIC_KEY } from '../decorators/public.decorator';

@Injectable()
export class TosGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (isPublic) return true;
    // ... rest of TOS check logic
  }
}
```

And in `auth.controller.ts`, decorate public routes:

```
import { Public } from '../common/decorators/public.decorator';

@Public()
@Post('register')
async register(@Body() body: RegisterDto) { ... }

@Public()
@Post('login')
async login(@Body() body: LoginDto) { ... }

@Public()
@Get('campuses')
async getCampuses(@Query('domain') domain: string) { ... }
```

Priority Fix Order

Priority	File	Fix
1 (CRITICAL)	apps/mobile/src/services/api.ts	Change localhost → 10.0.2.2:3000 (Android) or LAN IP

2 (CRITICAL)	apps/backend/src/modules/auth/dto/auth.dto.ts	Add tosAccepted field or remove from register payload
3 (HIGH)	apps/backend/src/common/guards/tos.guard.ts	Add @Public() bypass for auth routes
4 (MEDIUM)	apps/mobile/src/screens/auth/AuthScreen.tsx	Better error messages for campus/waitlist failures
5 (MEDIUM)	apps/mobile/src/store/useAuthStore.ts	Add AsyncStorage persistence

Fix #1 and #2 first — those are the two immediate causes of the total auth failure you're experiencing.